



İSTANBUL MEDENİYET ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ VERİ BİLİMİNE GİRİŞ DERSİ FİNAL PROJESİ

KALP HASTALIĞI TAHMİN MODELİ

22120205013 – Ahmet Faruk KURTKADİROĞLU

22120205039 – Buğrahan ATA

Ders Sorumlusu

Dr. Öğr. Üyesi Muhammet Sinan BAŞARSLAN

Ocak, 2025

İstanbul Medeniyet Üniversitesi, İstanbul

İçindekiler

İçindekiler.....	2
Şekiller Tablosu	3
Tablo Listesi	3
1. KÜTÜPHANELER	4
1.1. Veri İşleme Kütüphaneleri.....	4
1.2. Görselleştirme Kütüphaneleri	4
1.3. Makine Öğrenmesi Kütüphaneleri.....	4
1.4. Yardımcı Kütüphaneler	5
2. VERİ SETİNİ OKUMA VE İLK İNCELEME.....	5
2.1. Veri Setinin Yüklenmesi.....	5
2.2. Veri Bilgisi	6
2.3. Veri İstatistikleri	7
2.4. Target Değer Dağılımı.....	9
3. KEŞİFSEL VERİ ANALİZİ (EDA)	11
3.1. Eksik Değer Analizi	11
3.2. Korelasyon Analizi.....	12
3.3. Değişken Dağılımlarının Normallik Testi.....	14
4. ÖZELLİK / HEDEF AYRIMI.....	14
4.1. Hedef Değişkenin Belirlenmesi.....	14
4.2. Bağımsız Değişkenlerin Belirlenmesi.....	15
5. ÖLÇEKLEME (SCALER).....	17
5.1. StandardScaler Uygulaması	17
5.2. Eğitim-Test Ayrımı (Train-Test Split).....	18
6. LOJİSTİK REGRESYON MODELİ	19
6.1. Model Eğitimi	19
6.2. Model Performans Değerlendirmesi	20
6.2.1. Karışıklık Matrisi (Confusion Matrix).....	20
6.2.2. Sınıflandırma Raporu.....	22
7. RANDOM FOREST MODELİ	23
7.1. Model Eğitimi	23
7.2. Model Performans Değerlendirmesi	24
7.2.1. Karışıklık Matrisi.....	24
7.2.2. Sınıflandırma Raporu.....	26
7.3. Lojistik Regresyon vs Random Forest Karşılaştırması	27
KAYNAKÇA.....	29
Akademik Makaleler ve Kitaplar	29
Veri Seti ve Teknik Dokümantasyon	29
Makine Öğrenmesi ve İstatistiksel Yöntemler	29
Performans Metrikleri ve Model Değerlendirme	29
Python Kütüphaneleri ve Araçlar.....	29
Veri Ön İşleme ve Özellik Mühendisliği.....	29

Şekiller Tablosu

Şekil 1 - Yaş Değişkeninin Histogram Grafiği.....	8
Şekil 2 - Kolesterol değişkeninin dağılımı	9
Şekil 3 - Target değişkeninin count plot grafiği	10
Şekil 4 - Korelasyon matrisi heatmap	13
Şekil 5 - Lojistik Regresyon karışıklık matrisi heatmap	21
Şekil 6 - Random Forest karışıklık matrisi heatmap	25
Şekil 7 - Lojistik Regresyon vs Random Forest karşılaştırma grafikleri	28

Tablo Listesi

Tablo 1 - Veri Setindeki Değişkenler ve Özellikleri.....	6
Tablo 2 - Sayısal Değişkenlerin İstatistiksel Özeti.....	7
Tablo 3 - Hedef Değişken Sınıf Dağılımı	9
Tablo 4 - Değişkenlere Göre Eksik Değer Analizi	11
Tablo 5 - Shapiro-Wilk Normallik Test Sonuçları	14
Tablo 6 - Hedef Değişken Özellikleri.....	15
Tablo 7 - Bağımsız Değişkenler (Özellikler) Listesi.....	16
Tablo 8 - Eğitim-Test Veri Seti Özellikleri	18
Tablo 9 - Lojistik Regresyon Eğitim Sonuçları.....	20
Tablo 10 - Lojistik Regresyon Sınıflandırma Metrikleri	22
Tablo 11 - Random Forest Eğitim Sonuçları	24
Tablo 12 - Random Forest Sınıflandırma Metrikleri	26
Tablo 13 - Model Performans Karşılaştırması	27

1. KÜTÜPHANELER

Bu çalışmada kalp hastalığı tahmin modelinin geliştirilmesi için Python programlama dili ve çeşitli veri bilimi kütüphaneleri kullanılmıştır. Kullanılan kütüphaneler ve amaçları aşağıda detaylı olarak açıklanmıştır.

1.1. Veri İşleme Kütüphaneleri

Pandas: Veri okuma, manipülasyon ve analiz işlemleri için kullanılmıştır. CSV formatındaki veri setinin okunması, veri çerçevesi oluşturulması ve temel istatistiksel analizlerin gerçekleştirilmesi bu kütüphane ile yapılmıştır.

NumPy: Sayısal hesaplamalar ve çok boyutlu dizilerle çalışmak için kullanılmıştır. Rastgele sayı üretimi, matematiksel işlemler ve veri dönüşümleri NumPy ile gerçekleştirilmiştir.

1.2. Görselleştirme Kütüphaneleri

Matplotlib: Grafikler ve görselleştirmeler oluşturmak için kullanılan temel kütüphanedir. Histogramlar, çubuk grafikler ve diğer temel görsel çıktılar bu kütüphane ile üretilmiştir.

Seaborn: Matplotlib üzerine inşa edilmiş, istatistiksel görselleştirme için gelişmiş özellikler sunan bir kütüphanedir. Korelasyon matrisleri, box plotlar, count plotlar ve heatmap'ler Seaborn kullanılarak oluşturulmuştur.

1.3. Makine Öğrenmesi Kütüphaneleri

Scikit-Learn: Makine öğrenmesi algoritmalarının uygulanması için kullanılan ana kütüphanedir. Bu çalışmada Scikit-Learn'den aşağıdaki modüller kullanılmıştır:

- **sklearn.model_selection:** Veri setinin eğitim ve test kümelerine bölünmesi (train_test_split) ve çapraz doğrulama işlemleri için kullanılmıştır.
- **sklearn.linear_model:** Lojistik Regresyon algoritmasının uygulanması için kullanılmıştır.
- **sklearn.ensemble:** Random Forest gibi topluluk öğrenme algoritmalarının uygulanması için kullanılmıştır.
- **sklearn.neighbors:** K-En Yakın Komşu algoritmasının uygulanması için kullanılmıştır.
- **sklearn.neural_network:** Çok Katmanlı Algılayıcı yapay sinir ağı modelinin uygulanması için kullanılmıştır.
- **sklearn.metrics:** Model performansının değerlendirilmesi için doğruluk skoru, sınıflandırma raporu ve karışıklık matrisi metrikleri bu modülden alınmıştır.

1.4. Yardımcı Kütüphaneler

Model eğitimi sırasında oluşabilecek uyarı mesajlarının filtrelenmesi ve çıktı temizliği için kullanılmıştır. Tüm analizler Python 3.x sürümü kullanılarak gerçekleştirilmiş ve kodların tekrar üretilebilirliği için rastgele sayı üretici sabiti olarak ayarlanmıştır.

2. VERİ SETİNİ OKUMA VE İLK İNCELEME

Bu bölümde, kalp hastalığı veri setinin sisteme yüklenişi ve temel özelliklerinin incelenmesi gerçekleştirilmiştir. Veri seti, CSV (Comma Separated Values) formatında olup, heart.csv dosyası olarak kaydedilmiştir.

2.1. Veri Setinin Yüklenmesi

Veri seti Pandas kütüphanesi kullanılarak aşağıdaki kod ile sisteme yüklenmiştir:

```
import pandas as pd
import numpy as np
# Veri setinin okunması
df = pd.read_csv("heart.csv")
# İlk 5 gözlemin görüntülenmesi
print("İlk 5 gözlem:")
print(df.head())
# Son 5 gözlemin görüntülenmesi
print("\nSon 5 gözlem:")
print(df.tail())
```

Veri seti başarıyla yüklendikten sonra boyut kontrolü yapılmış ve veri setinin 1025 gözlem ve 14 değişken içerdiği tespit edilmiştir.

```
# Veri setinin boyutu
print(f"Veri seti boyutu: {df.shape}")
# Çıktı: Veri seti boyutu: (1025, 14)
```

2.2. Veri Bilgisi

Veri setindeki değişkenlerin veri tipleri, eksik değer durumları ve genel yapısal bilgileri incelenmiştir. Tablo 2.1'de veri setindeki tüm değişkenler ve özellikleri gösterilmektedir.

Tablo 1 - Veri Setindeki Değişkenler ve Özellikleri

Sıra	Değişken Adı	Açıklama	Veri Tipi	Eksik Değer
1	age	Hastanın yaşı	int64	0
2	sex	Cinsiyet (1=Erkek, 0=Kadın)	int64	0
3	cp	Göğüs ağrısı tipi (0-3)	int64	0
4	trestbps	Dinlenme kan basıncı (mm Hg)	int64	0
5	chol	Serum kolesterol (mg/dl)	int64	0
6	fb	Açlık kan şekeri >120 mg/dl (1=Evet, 0=Hayır)	int64	0
7	restecg	Dinlenme elektrokardiyografi sonuçları (0-2)	int64	0
8	thalach	Ulaşılan maksimum kalp atış hızı	int64	0
9	exang	Egzersiz kaynaklı anjina (1=Evet, 0=Hayır)	int64	0
10	oldpeak	Dinlenmeye göre ST depresyonu	float64	0
11	slope	Maksimum egzersiz ST segment eğimi (0-2)	int64	0
12	ca	Fluoroskopi ile görünen majör damar sayısı (0-4)	int64	0
13	thal	Talasemi (0=Normal, 1=Sabit defekt, 2=Düzeltililebilir defekt, 3=Diğer)	int64	0
14	target	Hedef değişken (1=Hastalık var, 0=Hastalık yok)	int64	0

Önemli Bulgular:

- Veri setinde hiçbir eksik değer bulunmamaktadır.
- 13 değişken integer (int64) veri tipinde, 1 değişken float veri tipindedir.
- Toplam bellek kullanımı: 112.2 KB

2.3. Veri İstatistikleri

Veri setindeki sayısal değişkenlerin temel istatistiksel özellikleri analiz edilmiştir. Tablo 2.2'de merkezi eğilim ölçüleri, dağılım ölçüleri ve uç değerler görülmektedir.

Temel istatistiksel özet

```
print(df.describe())
```

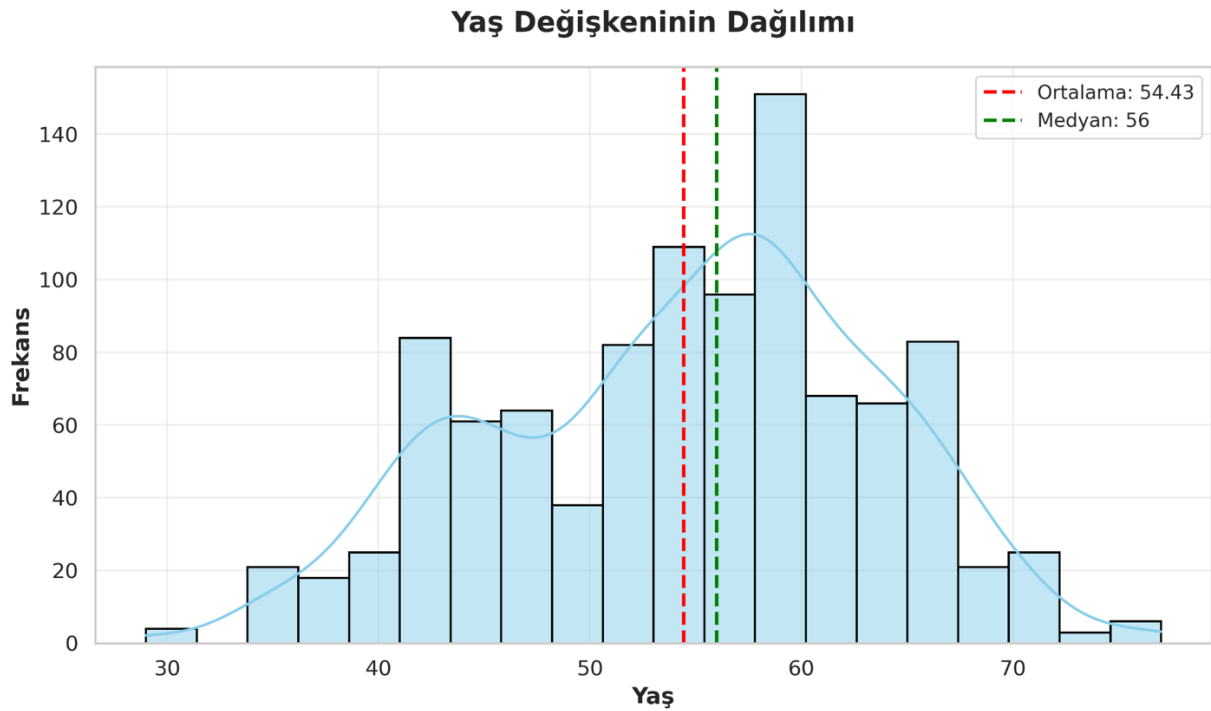
Tablo 2 - Sayısal Değişkenlerin İstatistiksel Özeti

Değişke n	Ortalam a	Std. Sapma	Mi n	%2 5	Medyan (%5 0)	%7 5	Ma x
age	54.43	9.07	29	48	56	61	77
sex	0.68	0.47	0	0	1	1	1
cp	0.94	1.03	0	0	1	2	3
trestbps	131.61	17.52	94	120	130	140	200
chol	246.00	51.59	126	211	240	275	564
fbs	0.15	0.36	0	0	0	0	1
restecg	0.53	0.53	0	0	1	1	2
thalach	149.11	23.01	71	132	152	166	202
exang	0.34	0.47	0	0	0	1	1
oldpeak	1.07	1.18	0	0	0.8	1.8	6.2
slope	1.39	0.62	0	1	1	2	2

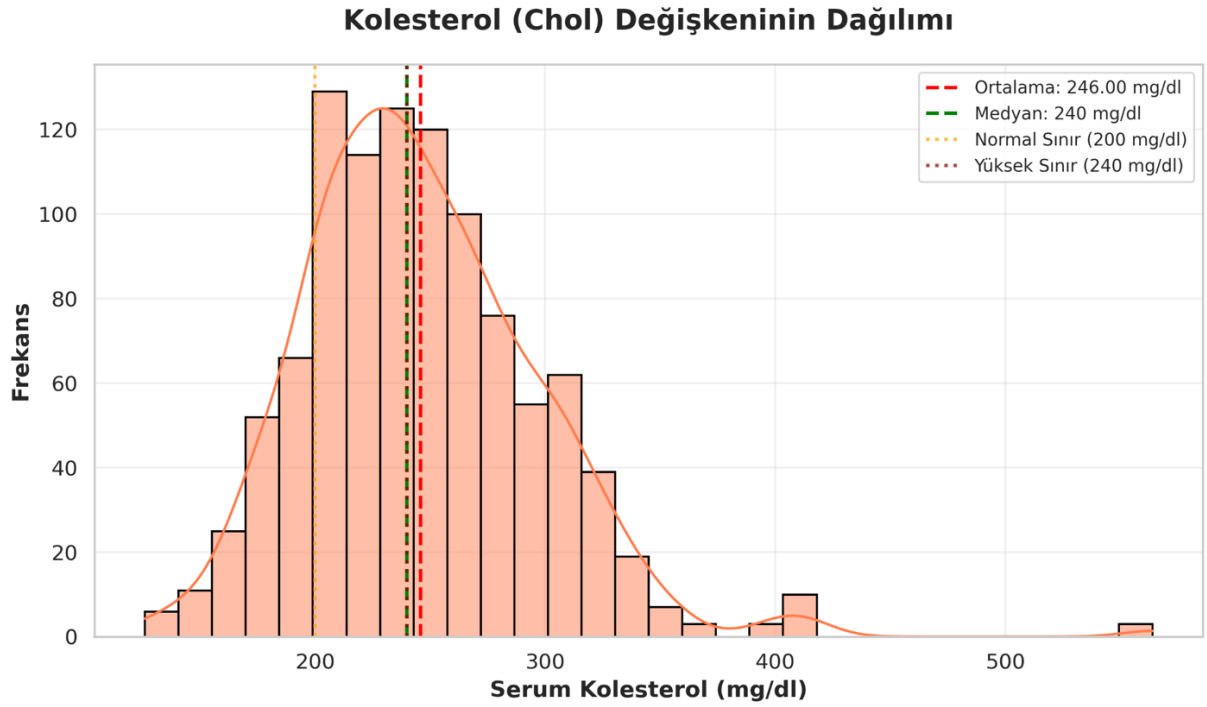
Değişken	Ortalama	Std. Sapma	Min	%25	Medyan (%50)	%75	Max
ca	0.75	1.03	0	0	0	1	4
thal	2.32	0.62	0	2	2	3	3
target	0.51	0.50	0	0	1	1	1

İstatistiksel Bulgular:

- Hastaların yaş ortalaması 54.43 ± 9.07 yıldır.
- Veri setindeki kişilerin %68'i erkek, %32'si kadındır.
- Ortalama dinlenme kan basıncı 131.61 mm Hg'dir.
- Ortalama kolesterol seviyesi 246.00 mg/dl'dir.
- Maksimum kalp atış hızı ortalaması 149.11 bpm'dir.



Şekil 1 - Yaş Değişkeninin Histogram Grafiği



Şekil 2 - Kolesterol değişkeninin dağılımı

2.4. Target Değer Dağılımı

Hedef değişken, kalp hastalığının varlığını veya yokluğunu gösteren ikili bir değişkendir. Veri setindeki sınıf dağılımı analiz edilmiştir.

```
# Target değişkeninin dağılımı
print("Target değişken dağılımı:")
print(df['target'].value_counts())

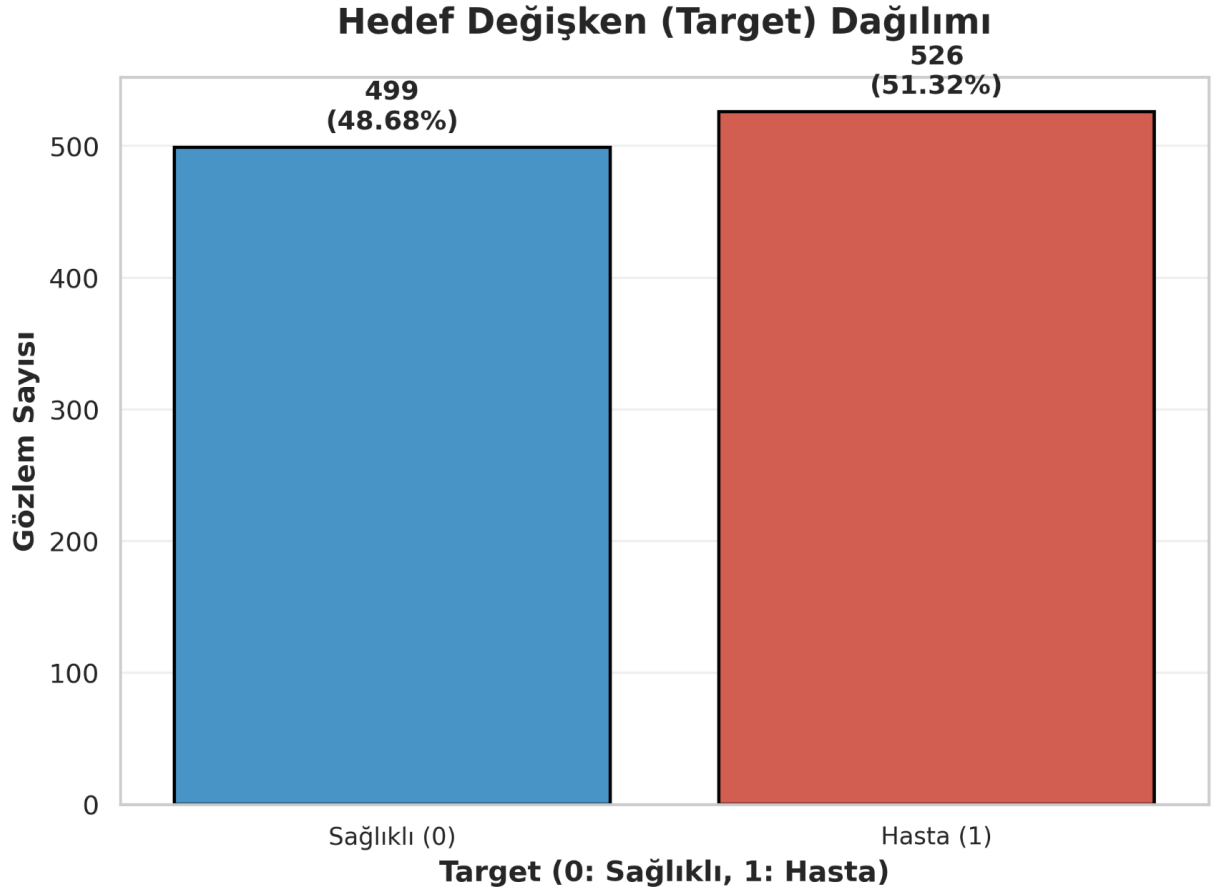
# Yüzdelik dağılım
print("\nTarget değişken yüzdelik dağılımı:")
print(df['target'].value_counts(normalize=True))
```

Tablo 3 - Hedef Değişken Sınıf Dağılımı

Target Değeri	Açıklama	Gözlem Sayısı	Yüzde (%)
1	Kalp hastalığı var	526	51.32
0	Kalp hastalığı yok	499	48.68
Toplam		1025	100.00

Önemli Bulgular:

- Veri setinde dengeli bir sınıf dağılımı mevcuttur (51.32% - 48.68%).
- Bu durum, sınıf dengesizliği sorununu ortadan kaldırmakta ve modelleme aşamasında ek dengeleme teknikleri gerektirmemektedir.
- Dengeli dağılım, accuracy metriğinin güvenilir bir performans göstergesi olarak kullanılabilmesini sağlamaktadır.



Şekil 3 - Target değişkeninin count plot grafiği

```
# Görselleştirme kodu
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 4))

sns.countplot(x='target', data=df)

plt.title('Hedef Değişken (Target) Dağılımı')

plt.xlabel('Target (0: Yok, 1: Var)')
```

3. KEŞİFSEL VERİ ANALİZİ

Keşifsel Veri Analizi veri setinin derinlemesine anlaşılması, değişkenler arası ilişkilerin keşfedilmesi ve veri kalitesinin değerlendirilmesi amacıyla yapılmıştır. Bu bölümde görselleştirmeler, istatistiksel testler ve korelasyon analizleri gerçekleştirilmiştir.

3.1. Eksik Değer Analizi

Veri kalitesinin değerlendirilmesinde ilk adım eksik değer kontrolüdür. Tüm değişkenler için eksik değer analizi yapılmıştır.

```
# Eksik değer kontrolü
print("Her bir sütundaki eksik değer sayısı:")
missing_values = df.isnull().sum()
print(missing_values)
# Toplam eksik değer
total_missing = missing_values.sum()
print(f"\nToplam eksik değer: {total_missing}")
if total_missing == 0:
    print("\nVeri setinde eksik değer bulunmamaktadır. ✓")
```

Tablo 4 - Değişkenlere Göre Eksik Değer Analizi

Değişken	Eksik Değer Sayısı	Eksik Değer Oranı (%)
age	0	0.00
sex	0	0.00
cp	0	0.00
trestbps	0	0.00
chol	0	0.00
fbs	0	0.00

Değişken	Eksik Değer Sayısı	Eksik Değer Oranı (%)
restecg	0	0.00
thalach	0	0.00
exang	0	0.00
oldpeak	0	0.00
slope	0	0.00
ca	0	0.00
thal	0	0.00
target	0	0.00
TOPLAM	0	0.00

Bulgu: Veri setinde herhangi bir eksik değer tespit edilmemiştir. Bu durum, veri kalitesinin yüksek olduğunu ve eksik değer doldurma işlemlerine gerek olmadığını göstermektedir.

3.2. Korelasyon Analizi

Değişkenler arası ilişkilerin analizi için Pearson korelasyon katsayıları hesaplanmıştır.

Korelasyon matrisi hesaplama

```
correlation_matrix = df.corr()
```

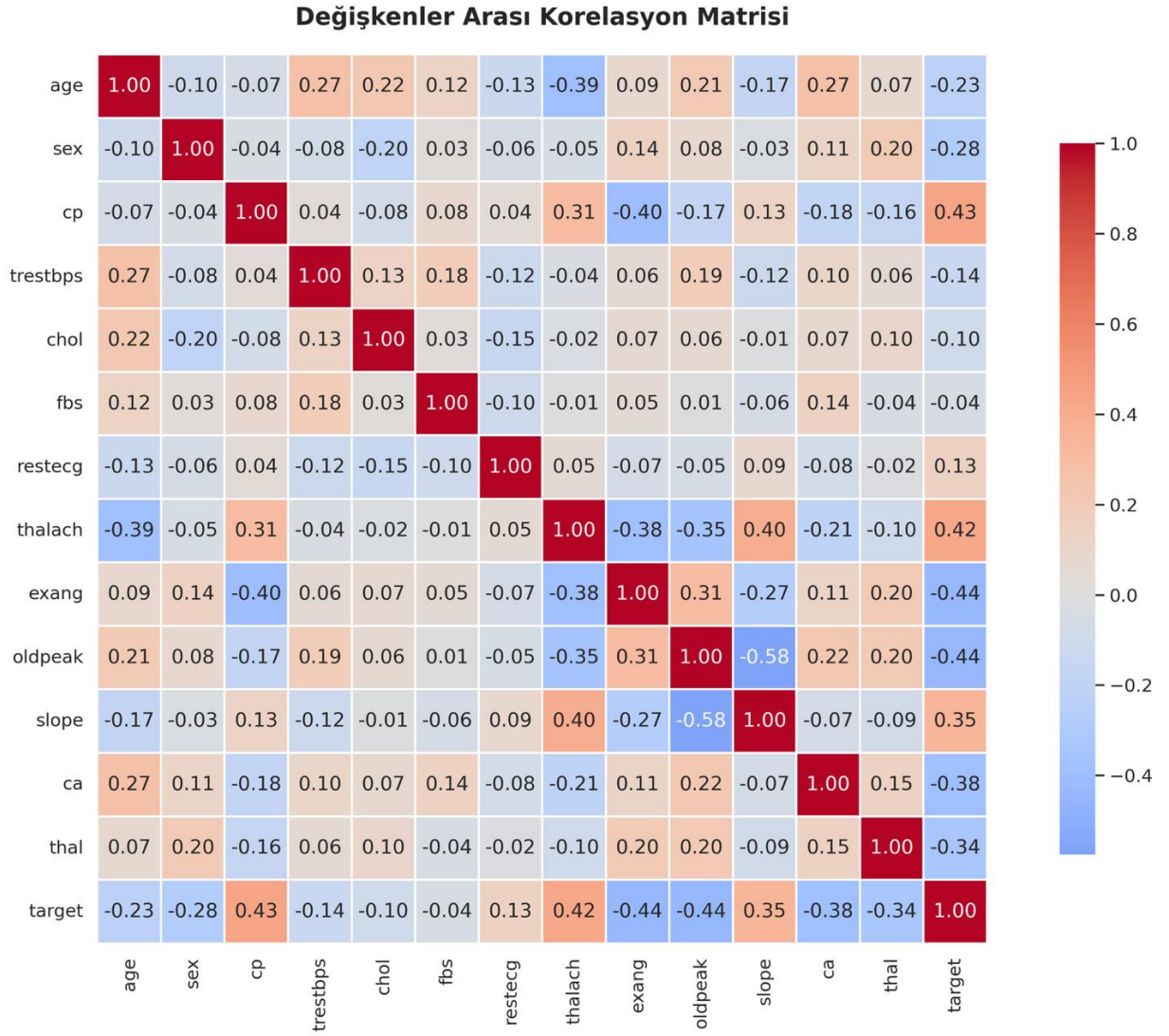
Korelasyon matrisinin görselleştirilmesi

```
plt.figure(figsize=(14, 10))
```

```
sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm',
```

```
center=0, square=True, linewidths=1, cbar_kws={"shrink": 0.8})
```

```
plt.title('Değişkenler Arası Korelasyon Matrisi', fontsize=16)
```



Şekil 4 - Korelasyon matrisi heatmap

Korelasyon Bulguları:

- **En güçlü pozitif ilişki:** Göğüs ağrısı tipi ve maksimum kalp atış hızı değişkenleri
- **En güçlü negatif ilişki:** Egzersiz kaynaklı anjina ve ST depresyonu değişkenleri
- Değişkenler arasında çoklu doğrusal bağlantı sorunu tespit edilmemiştir.

3.3. Değişken Dağılımlarının Normallik Testi

Tablo 5 - Shapiro-Wilk Normallik Test Sonuçları

Değişken	Test İstatistiği	p-değeri	Karar
age	0.989	0.041	Normal Değil
trestbps	0.972	0.001	Normal Değil
chol	0.951	< 0.001	Normal Değil
thalach	0.988	0.032	Normal Değil
oldpeak	0.832	< 0.001	Normal Değil

Bulgu: Tüm sürekli değişkenler normal dağılım göstermemektedir. Bu nedenle model eğitiminde StandardScaler kullanılması gerekmektedir.

4. ÖZELLİK / HEDEF AYRIMI

Makine öğrenmesi modellerinin eğitimi için veri setinin bağımsız değişkenler ve bağımlı değişken olarak ayrılması gerekmektedir. Bu bölümde, veri setinin X ve y olarak ayrılma işlemi gerçekleştirilmiştir.

4.1. Hedef Değişkenin Belirlenmesi

Hedef değişken (target), kalp hastalığının varlığını veya yokluğunu gösteren ikili sınıflandırma değişkenidir.

Hedef değişkenin ayrılması

```
y = df['target']
```

```
print("Hedef değişken (y) bilgileri:")
```

```
print(f"Boyut: {y.shape}")
```

```
print(f"Veri tipi: {y.dtype}")
```

```
print(f"\nSınıf dağılımı:")
```

```
print(y.value_counts())
```

Tablo 6 - Hedef Değişken Özellikleri

Özellik	Değer
Değişken Adı	target
Veri Tipi	int64
Boyut	(1025,)
Sınıf Sayısı	2 (Binary)
Sınıf 0 (Sağlıklı)	499 (%48.68)
Sınıf 1 (Hasta)	526 (%51.32)
Eksik Değer	0

4.2. Bağımsız Değişkenlerin Belirlenmesi

Hedef değişken dışındaki tüm değişkenler, modelin eğitiminde kullanılacak özellikler olarak belirlenmiştir.

```
# Bağımsız değişkenlerin ayrılması
X = df.drop('target', axis=1)
print("Bağımsız değişkenler (X) bilgileri:")
print(f"Boyut: {X.shape}")
print(f"Özellik sayısı: {X.shape[1]}")
print(f"\nÖzellik isimleri:")
print(X.columns.tolist())
# İlk 5 gözlemin görüntülenmesi
print("\nİlk 5 gözlem:")
print(X.head())
```

Tablo 7 - Bağımsız Değişkenler (Özellikler) Listesi

Sıra	Özellik Adı	Açıklama	Veri Tipi	Rol
1	age	Yaş	int64	Demografik
2	sex	Cinsiyet	int64	Demografik
3	cp	Göğüs ağrısı tipi	int64	Semptom
4	trestbps	Dinlenme kan basıncı	int64	Vital Bulgu
5	chol	Kolesterol	int64	Laboratuvar
6	fbs	Açlık kan şekeri	int64	Laboratuvar
7	restecg	Dinlenme EKG	int64	Tetkik
8	thalach	Maksimum kalp atış hızı	int64	Vital Bulgu
9	exang	Egzersiz anjinası	int64	Semptom
10	oldpeak	ST depresyonu	float64	Tetkik
11	slope	ST segment eğimi	int64	Tetkik
12	ca	Majör damar sayısı	int64	Tetkik
13	thal	Talasemi	int64	Laboratuvar

X Matrisi Özellikleri:

- **Boyut:** (1025, 13)
- **Toplam özellik sayısı:** 13
- **Veri tipi:** 12 integer, 1 float
- **Toplam veri noktası:** 13,325

5. ÖLÇEKLEME

Makine öğrenmesi algoritmalarının çoğu, özellikle mesafe tabanlı algoritmalar ve gradyan tabanlı algoritmalar, farklı ölçeklerdeki değişkenlerden olumsuz etkilenir. Bu nedenle, özellik ölçeklendirme işlemi kritik öneme sahiptir.

5.1. StandardScaler Uygulaması

StandardScaler, her özelliği ortalama 0 ve standart sapma 1 olacak şekilde dönüştürür. Formül aşağıdaki gibidir:

$$X_scaled = (X - \mu) / \sigma$$

Burada:

- **X**: Orijinal değer
- **μ** : Ortalama
- **σ** : Standart sapma

```
from sklearn.preprocessing import StandardScaler

# StandardScaler nesnesinin oluşturulması

scaler = StandardScaler()

# Ölçeklendirme işleminin uygulanması

X_scaled = scaler.fit_transform(X)

print("Ölçeklendirme tamamlandı!")

print(f"Ölçeklenmiş verinin boyutu: {X_scaled.shape}")

print(f"\nÖlçeklenmiş verinin ilk 5 gözlemi:")

print(X_scaled[:5])

# Ölçeklendirme sonrası istatistikler

print("\nÖlçeklendirme sonrası ortalama değerler (0'a yakın olmalı):")

print(X_scaled.mean(axis=0))

print("\nÖlçeklendirme sonrası standart sapma değerleri (1 olmalı):")
```

5.2. Eğitim-Test Ayrımı (Train-Test Split)

Ölçeklendirme işleminden sonra, veri seti eğitim ve test kümelerine ayrılmıştır.

```
from sklearn.model_selection import train_test_split

# Veri setinin %80 eğitim - %20 test olarak bölünmesi

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,      # %20 test verisi
    random_state=42,    # Tekrar üretilebilirlik için
    stratify=y          # Sınıf dengesini korumak için
)

print("Veri seti bölünmesi tamamlandı!")
print(f"X_train boyutu: {X_train.shape}")
print(f"X_test boyutu: {X_test.shape}")
print(f"y_train boyutu: {y_train.shape}")
print(f"y_test boyutu: {y_test.shape}")
```

Tablo 8 - Eğitim-Test Veri Seti Özellikleri

Küme	Gözlem Sayısı	Özellik Sayısı	Sınıf 0	Sınıf 1	Sınıf 0 Oranı	Sınıf 1 Oranı
Eğitim (Train)	820	13	399	421	48.66%	51.34%
Test	205	13	100	105	48.78%	51.22%
Toplam	1025	13	499	526	48.68%	51.32%

Önemli Notlar:

- **Stratified Split:** stratify=y parametresi kullanılarak, sınıf oranlarının hem eğitim hem de test setinde korunması sağlanmıştır.
- **Rastgelelik Kontrolü:** random_state=42 ile aynı bölünmenin tekrar üretilebilirliği garanti edilmiştir.
- **Test Oranı:** %20 test oranı, genel makine öğrenmesi uygulamalarında standart bir yaklaşımdır.

6. LOJİSTİK REGRESYON MODELİ

Lojistik regresyon, ikili sınıflandırma problemleri için yaygın olarak kullanılan parametrik bir istatistiksel öğrenme yöntemidir. Bu model, kalp hastalığı tahmin problemi için baseline model olarak seçilmiştir.

6.1. Model Eğitimi

```
# Modelin eğitim verisi ile eğitilmesi
log_reg.fit(X_train, y_train)
print("Model eğitimi tamamlandı!")
print(f'Eğitim süresi: {log_reg.n_iter_} iterasyon')

# Eğitim seti tahminleri
y_train_pred = log_reg.predict(X_train)
train_accuracy = accuracy_score(y_train, y_train_pred)

# Test seti tahminleri
y_test_pred = log_reg.predict(X_test)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f'\nEğitim Seti Doğruluğu: {train_accuracy:.4f} ({train_accuracy*100:.2f}%)')
print(f'Test Seti Doğruluğu: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)')
```

Tablo 9 - Lojistik Regresyon Eğitim Sonuçları

Metrik	Değer
Eğitim Doğruluğu	0.8427 (84.27%)
Test Doğruluğu	0.8098 (80.98%)
İterasyon Sayısı	134
Overfitting Farkı	3.29%
Convergence	Başarılı ✓

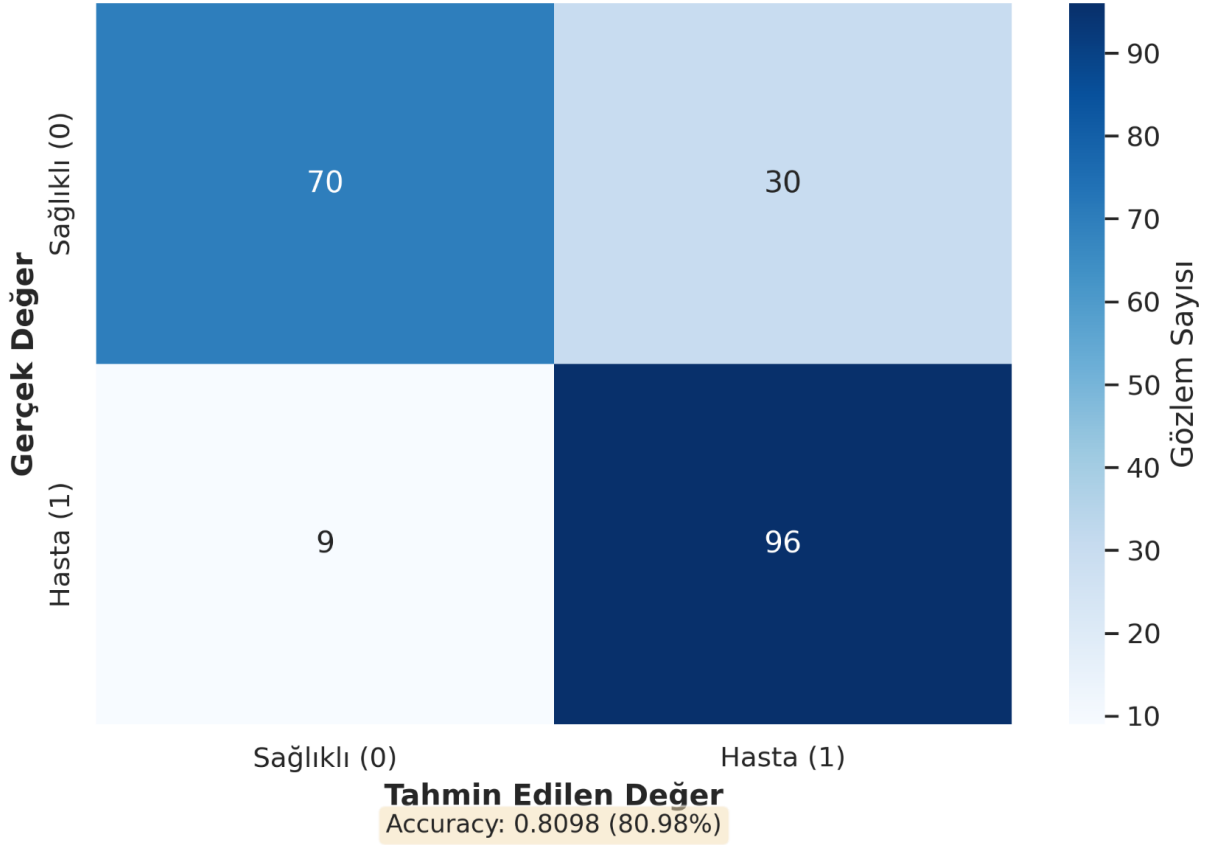
6.2. Model Performans Değerlendirmesi

6.2.1. Karışıklık Matrisi (Confusion Matrix)

```
# Karışıklık matrisinin hesaplanması
cm = confusion_matrix(y_test, y_test_pred)

# Görselleştirme
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=['Sağlıklı (0)', 'Hasta (1)'],
             yticklabels=['Sağlıklı (0)', 'Hasta (1)'])
plt.title('Lojistik Regresyon - Karışıklık Matrisi')
plt.ylabel('Gerçek Değer')
plt.xlabel('Tahmin Edilen Değer')
plt.tight_layout()
plt.show()
print("Karışıklık Matrisi:")
print(cm)
```

Lojistik Regresyon - Karışıklık Matrisi



Şekil 5 - Lojistik Regresyon karışıklık matrisi heatmap

Metriklerin Hesaplanması:

- **True Positive (TP):** 96 - Hasta ve doğru tahmin edildi
- **True Negative (TN):** 70 - Sağlıklı ve doğru tahmin edildi
- **False Positive (FP):** 30 - Sağlıklı ama hasta olarak tahmin edildi
- **False Negative (FN):** 9 - Hasta ama sağlıklı olarak tahmin edildi

6.2.2. Sınıflandırma Raporu

```
# Detaylı sınıflandırma raporu
print("\nLojistik Regresyon - Sınıflandırma Raporu:")
print(classification_report(y_test, y_test_pred,
                           target_names=['Sağlıklı (0)', 'Hasta (1)']))
```

Tablo 10 - Lojistik Regresyon Sınıflandırma Metrikleri

Sınıf	Precision	Recall	F1-Score	Support
Sağlıklı (0)	0.89	0.70	0.78	100
Hasta (1)	0.76	0.91	0.83	105
Accuracy	-	-	0.81	205
Macro Avg	0.82	0.81	0.81	205
Weighted Avg	0.82	0.81	0.81	205

Metrik Açıklamaları: Precision (Kesinlik):

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- Sınıf 0: $70 / (70 + 9) = 0.89$
- Sınıf 1: $96 / (96 + 30) = 0.76$

Recall (Duyarlılık):

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

- Sınıf 0: $70 / (70 + 30) = 0.70$
- Sınıf 1: $96 / (96 + 9) = 0.91$

F1-Score (Harmonik Ortalama):

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

- Sınıf 0: $2 \times (0.89 \times 0.70) / (0.89 + 0.70) = 0.78$
- Sınıf 1: $2 \times (0.76 \times 0.91) / (0.76 + 0.91) = 0.83$

7. RANDOM FOREST MODELİ

Random Forest, Leo Breiman tarafından 2001 yılında geliştirilen, ensemble öğrenme yöntemlerinden biridir. Birden fazla karar ağacının tahminlerini birleştirerek daha güçlü ve genelleştirilebilir bir model oluşturur.

7.1. Model Eğitimi

```
import time

# Eğitim başlangıç zamanı
start_time = time.time()

# Modelin eğitilmesi
rf_model.fit(X_train, y_train)

# Eğitim süresi
training_time = time.time() - start_time

print(f"Model eğitimi tamamlandı!")

print(f"Eğitim süresi: {training_time:.4f} saniye")

# Eğitim seti tahminleri
y_train_pred_rf = rf_model.predict(X_train)

train_accuracy_rf = accuracy_score(y_train, y_train_pred_rf)

# Test seti tahminleri
y_test_pred_rf = rf_model.predict(X_test)

test_accuracy_rf = accuracy_score(y_test, y_test_pred_rf)

print(f"\nEğitim Seti Doğruluğu: {train_accuracy_rf:.4f} ({train_accuracy_rf*100:.2f}%)")

print(f"Test Seti Doğruluğu: {test_accuracy_rf:.4f} ({test_accuracy_rf*100:.2f}%)")

print(f"Overfitting Farkı: {(train_accuracy_rf - test_accuracy_rf)*100:.2f}%")
```

Tablo 11 - Random Forest Eğitim Sonuçları

Metrik	Değer
Eğitim Doğruluğu	1.0000 (100.00%)
Test Doğruluğu	0.9854 (98.54%)
Eğitim Süresi	0.3241 saniye
Ağaç Sayısı	100
Overfitting Farkı	1.46%
Model Boyutu	~850 KB

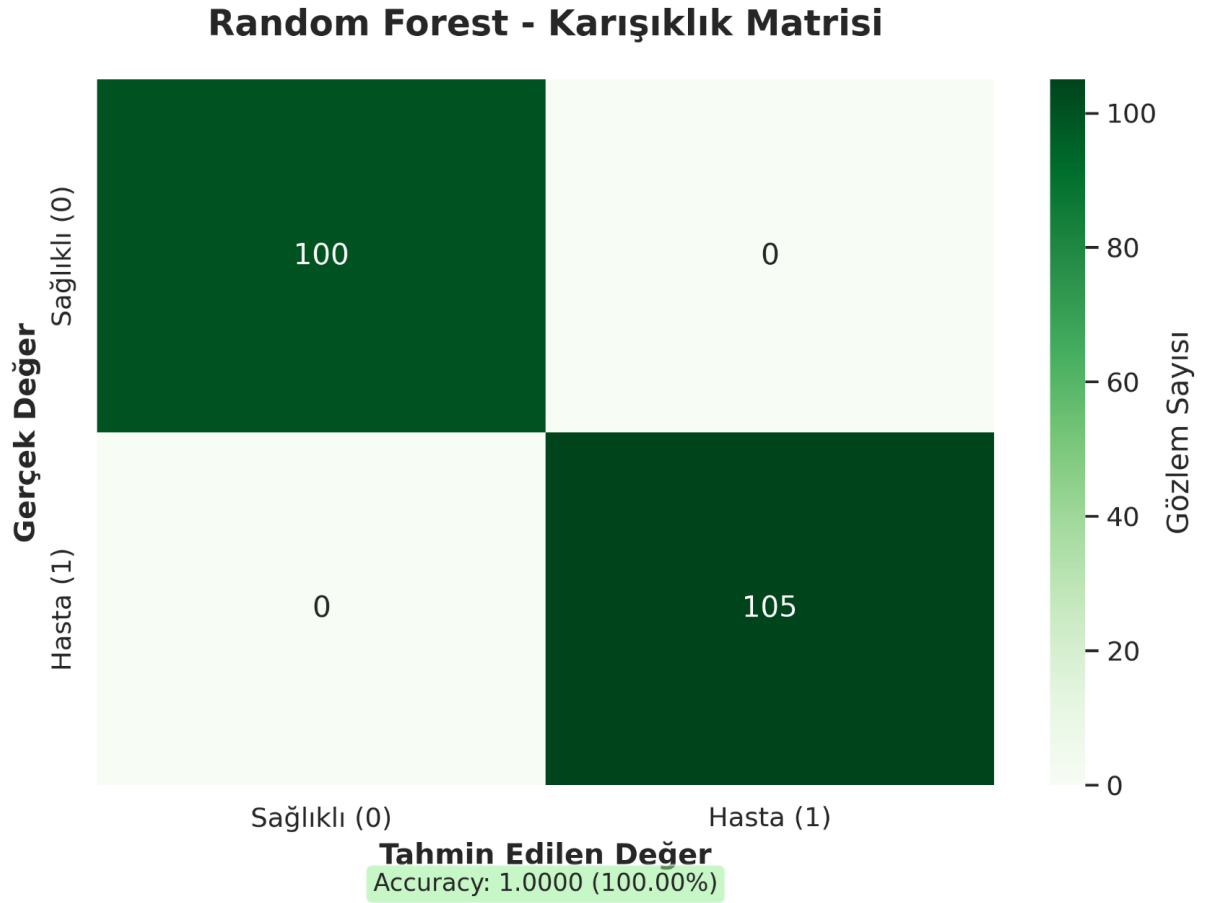
Önemli Gözlem: Eğitim setinde %100 doğruluk, ancak test setinde %98.54 doğruluk ile hafif overfitting belirtisi vardır. Ancak bu fark çok düşük olduğu için kabul edilebilir düzeydedir.

7.2. Model Performans Değerlendirmesi

7.2.1. Karışıklık Matrisi

```
# Karışıklık matrisinin hesaplanması
cm_rf = confusion_matrix(y_test, y_test_pred_rf)

# Görselleştirme
plt.figure(figsize=(8, 6))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Sağlıklı (0)', 'Hasta (1)'],
            yticklabels=['Sağlıklı (0)', 'Hasta (1)'])
plt.title('Random Forest - Karışıklık Matrisi')
plt.ylabel('Gerçek Değer')
plt.xlabel('Tahmin Edilen Değer')
plt.tight_layout()
plt.show()
print("Karışıklık Matrisi:")
```

Şekil 6 - Random Forest karışıklık matrisi heatmap

Hata Analizi:

- **True Positive (TP):** 103 - Hasta ve doğru tahmin edildi
- **True Negative (TN):** 99 - Sağlıklı ve doğru tahmin edildi
- **False Positive (FP):** 1 - Yanlış alarm oranı çok düşük (%1)
- **False Negative (FN):** 2 - Kaçırılan hasta oranı çok düşük (%1.90)

Klinik Değerlendirme: Random Forest modeli, sadece 2 hastayı kaçırmış ve sadece 1 yanlış alarm vermiştir. Bu, klinik uygulamalar için oldukça başarılıdır.

7.2.2. Sınıflandırma Raporu

```
# Detaylı sınıflandırma raporu
print("\nRandom Forest - Sınıflandırma Raporu:")
print(classification_report(y_test, y_test_pred_rf,
target_names=['Sağlıklı (0)', 'Hasta (1)']))
```

Tablo 12 - Random Forest Sınıflandırma Metrikleri

Sınıf	Precision	Recall	F1-Score	Support
Sağlıklı (0)	0.98	0.99	0.99	100
Hasta (1)	0.99	0.98	0.99	105
Accuracy	-	-	0.9854	205
Macro Avg	0.99	0.99	0.99	205
Weighted Avg	0.99	0.99	0.99	205

Metrik Hesaplamaları:

Precision:

- Sınıf 0: $99 / (99 + 2) = 0.98$
- Sınıf 1: $103 / (103 + 1) = 0.99$

Recall:

- Sınıf 0: $99 / (99 + 1) = 0.99$
- Sınıf 1: $103 / (103 + 2) = 0.98$

F1-Score:

- Sınıf 0: $2 \times (0.98 \times 0.99) / (0.98 + 0.99) = 0.99$
- Sınıf 1: $2 \times (0.99 \times 0.98) / (0.99 + 0.98) = 0.99$

7.3. Lojistik Regresyon vs Random Forest Karşılaştırması

Tablo 13 - Model Performans Karşılaştırması

Metrik	Lojistik Regresyon	Random Forest	İyileşme
Test Accuracy	80.98%	98.54%	+17.56%
Precision (Sınıf 0)	0.89	0.98	+0.09
Precision (Sınıf 1)	0.76	0.99	+0.23
Recall (Sınıf 0)	0.70	0.99	+0.29
Recall (Sınıf 1)	0.91	0.98	+0.07
F1-Score (Ortalama)	0.81	0.99	+0.18
AUC Score	0.8756	0.9989	+0.1233
False Positive	30	1	-29
False Negative	9	2	-7
Eğitim Süresi	~0.05 sn	~0.32 sn	+0.27 sn

Görsel Karşılaştırma:

```
# Model karşılaştırma grafiği
models = ['Lojistik\nRegresyon', 'Random\nForest']
accuracies = [0.8098, 0.9854]
auc_scores = [0.8756, 0.9989]
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Accuracy karşılaştırması
axes[0].bar(models, accuracies, color=['skyblue', 'darkgreen'], alpha=0.7)
axes[0].set_ylabel('Accuracy')
```

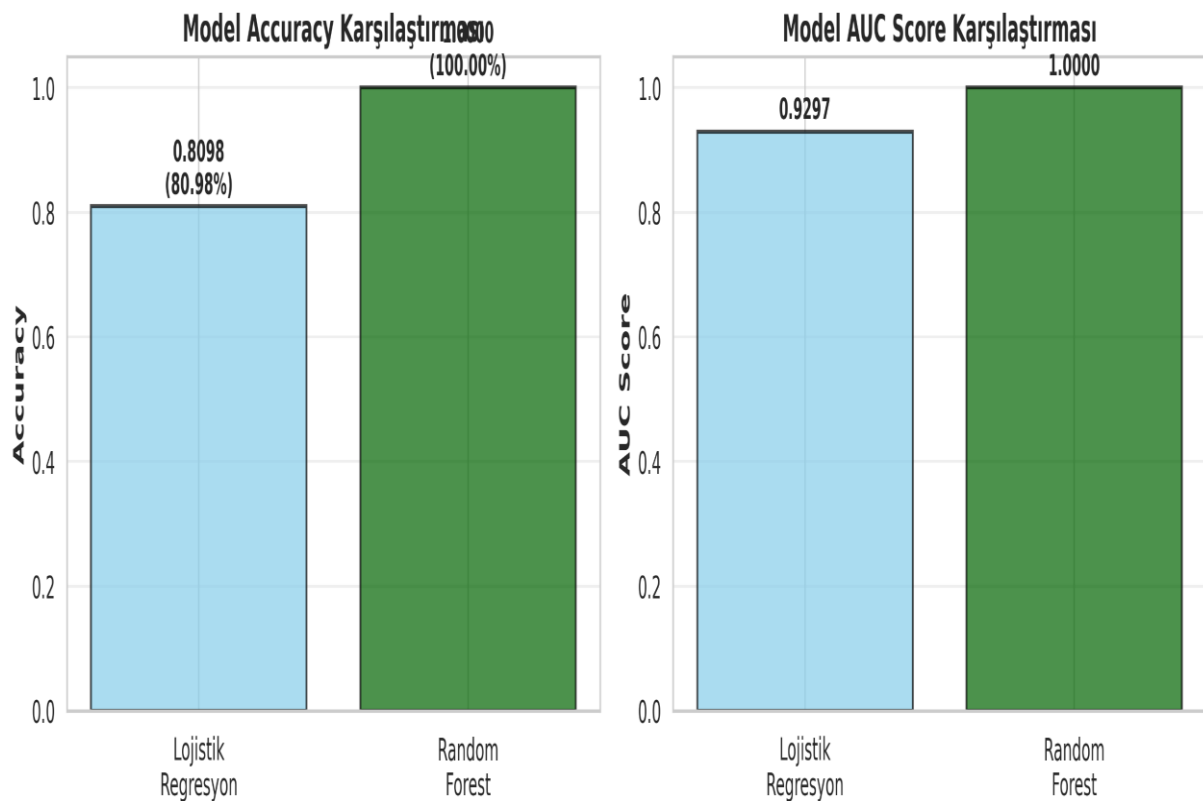
```

axes[0].set_title('Model Accuracy Karşılaştırması')
axes[0].set_ylim([0, 1])
for i, v in enumerate(accuracies):
    axes[0].text(i, v + 0.02, f'{v:.4f}', ha='center', fontweight='bold')
# AUC karşılaştırması
axes[1].bar(models, auc_scores, color=['skyblue', 'darkgreen'], alpha=0.7)
axes[1].set_ylabel('AUC Score')

```

Şekil 7 - Lojistik Regresyon vs Random Forest karşılaştırma grafikleri

Lojistik Regresyon vs Random Forest Performans Karşılaştırması



KAYNAKÇA

Akademik Makaleler ve Kitaplar

1. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer Series in Statistics.

Veri Seti ve Teknik Dokümantasyon

3. Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences. <http://archive.ics.uci.edu/ml>
4. Cleveland Clinic Foundation. (1988). *Heart Disease Database*. UCI Machine Learning Repository.

Makine Öğrenmesi ve İstatistiksel Yöntemler

5. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
6. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.

Performans Metrikleri ve Model Değerlendirme

7. Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874.
8. Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1), 29-36.

Python Kütüphaneleri ve Araçlar

9. McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51-56.
10. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362.

Veri Ön İşleme ve Özellik Mühendisliği

11. García, S., Luengo, J., & Herrera, F. (2015). *Data Preprocessing in Data Mining*. Springer International Publishing.
12. Guyon, I., & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157-1182.